

SecureLock

User manual: running and demonstrating the prototype

OPERATOR'S GUIDE

**Start it, drive it, and show
somebody why it works.**

Every command in this manual has been run on a clean machine.

This manual assumes you can open a terminal and type a command, and nothing beyond that. Part A gets the system running. Part B is a guided demonstration you can perform in front of an audience. Part C documents every screen. Part D is what to do when something breaks.

Part A — Getting it running

A1. What you need

Requirement	Notes
Node.js 18 or newer	Runs the blockchain and the web app.
Python 3.11, 3.12 or 3.13	Not 3.14. A required library cannot be built on 3.14 and pip fails with a long Rust error. On a Mac: <code>brew install python@3.13</code>
Nothing else	No internet, no API keys, no database server, no Docker. The whole system runs offline on one machine.

A2. Start everything with one command

From the project folder:

```
./scripts/start.sh      # macOS and Linux
.\scripts\start.ps1    # Windows
```

The first run takes a few minutes because it installs dependencies and seeds the demo data. Later runs take seconds. The script checks your Python version before doing anything, reuses any service that is already running, and prints the addresses when it is done.

A3. What is now running

Three services. All three are needed; stopping any one breaks the system in a visible way.

Service	Address	If you stop it
Blockchain	127.0.0.1:8545	The seal goes dark, the countdown stops, and release is refused rather than falling back to a server clock.
API	127.0.0.1:8000	Every signed-in page shows nothing.
Web app	localhost:3000	The site itself is gone.

The API also publishes its own interactive documentation at `127.0.0.1:8000/docs`, where every endpoint can be called directly. Useful for proving that the web app is not faking anything.

A4. Signing in

Open `http://localhost:3000`. The landing page needs no account. Click **Open the register**, then click any demo account row to fill the form, and press **Sign in**.

The password for every demo account is `SecureLock#2026`. It is deliberately obvious: this is local demonstration data, not a real credential.

Account	Role	Use it to show
authority@securelock.demo	Exam Authority	Assembling, sealing and releasing papers
setter1@securelock.demo	Question Setter	How little a setter can see
reviewer@securelock.demo	Reviewer	Approving without ever being able to edit
auditor@securelock.demo	Auditor	Verifying fingerprints without reading questions
admin@securelock.demo	Super Admin	Managing users — and the tamper demonstration

Part B — The guided demonstration

Eight beats, about ten minutes. Beat 3 is the one to spend time on; everything else is context for it.

Beat 1 — The landing page, before signing in

Do: open `http://localhost:3000` and point at the dark panel.

Say: that panel is not reading our server. It is reading the blockchain directly from the browser. The top clock is the chain's own time and the block number climbs while you watch. The bottom clock is this laptop's, printed and struck through, because nothing in the system consults it.

Beat 2 — A paper under seal

Do: sign in as **authority**. The dashboard opens.

Say: the dark block is a real sealed paper, with its fingerprint and the hour it opens. Everything else on the page is quiet; the sealed object is the only thing rendered in ink, because sealed means you cannot see inside.

Beat 3 — Attack the clock (the important one)

Do: open **Time Lock**. Press **Year 2099**, then press **Attempt decryption**.

Expect: a refusal, stating how many seconds remain according to blockchain time.

What to say here

"I have just told the application that it is the year 2099. It believed me — look, the device clock says 2099. And the answer is still no, because the contract never asked this computer what time it was. That button calls the real decryption endpoint, not a mock. The refusal has just been written into the audit trail."

Beat 4 — Permission is real, not hidden buttons

Do: sign out, sign in as **setter1**, and look at the navigation. Open **Question Vault** and try to open a question written by another setter.

Expect: a refusal, which is itself recorded.

Say: the setter is not simply being shown fewer buttons. The server checks permission on every request, so typing the address directly fails in exactly the same way.

Beat 5 — Review without the power to edit

Do: sign in as **reviewer** and open **Review Queue**. Approve a question.

Say: READ, WRITE and APPROVE are three independent permissions. A reviewer holds two of them and never the third. Once approved, a question is frozen — including against the person who wrote it.

Beat 6 — Assemble a paper

Do: back as **authority**, open **Paper Builder** and generate a paper.

Say: nobody chose these questions by hand. The engine balanced the blueprint, avoided near-duplicates, and spread the authorship across setters — so no setter knows which of their questions was used, or alongside what.

Beat 7 — Break the audit trail on purpose

Do: sign in as **admin**, open **Audit Trail**, confirm the chain reports intact, then use the tamper demonstration to alter one historical row.

Expect: **TAMPERING DETECTED**, naming the broken link. The same screen restores it afterwards.

Say: each entry carries the fingerprint of the one before it. Editing a single line breaks every link after it, so a quiet correction is impossible.

Beat 8 — Show that the chain is real

Do: open **Blockchain**.

Say: the left column is what we submitted; the right column is read back off the chain itself, not from our database. Any transaction that failed is shown as failed, with the contract's own reason. Nothing here is invented — when the chain cannot be reached, the system records that instead of producing a hash that would look convincing and mean nothing.

The automated version of all of this

If you would rather have the whole flow asserted rather than clicked, one command runs all of it and checks every outcome:

```
backend/.venv/bin/python scripts/e2e_demo.py
```

It finishes with **ALL CHECKS PASSED** after 56 assertions, including the 2099 clock attack. The two test suites are:

```
cd blockchain && npx hardhat test                # 37 contract tests
cd backend && .venv/bin/python -m pytest tests    # 44 backend tests
```

Part C — Every screen

The navigation changes with the role, because each role is given a genuinely different system. Not every account sees every entry below.

Screen	What it is for
Landing page	Public. The live seal, reading the chain directly from the browser without signing in.
Dashboard	The state of the register: what is under seal, how many questions exist, how many access attempts were refused, and the most recent entries.
Question Vault	Every question as metadata and fingerprint only. Opening one requires permission and is recorded.
Create Question	Write a new question. It is encrypted and fingerprinted before storage, and anchored on the chain.
Review Queue	Questions awaiting approval. Approve or reject; never edit.
Variations	Rule-based rewordings of a question, gated behind a reviewer. Restates the instruction without touching the substance, which is why the expected answer cannot drift.
Paper Builder	Assemble a paper against the blueprint. Shows compliance, duplicate risk, per-question selection reasoning and contributor spread.
Papers	Every paper and its lifecycle status: draft, encrypted, registered, sealed, released.
Time Lock	The demonstration. Two clocks, the clock-attack presets, and a button that calls the real decryption endpoint.
Audit Trail	Every event, hash-chained. Includes the integrity check and the tamper demonstration.
Blockchain	Submitted transactions beside events read back from the chain, plus the deployed contract addresses.
Security	The posture summary: refusals, permission grants and integrity checks in one place.

Part D — When something goes wrong

D1. Resetting the demo

Read this before you reset anything

Reset the database and the chain **together**, using `./scripts/reset_demo.sh`. Never reset just one.

Identifiers like PAPER-2026-001 are regenerated from the start when the database is reseeded, but the chain still holds the previous run's records under those same identifiers. The contracts then refuse everything that follows — and most visibly, **no new paper can ever be sealed again**. The contracts are right to refuse. The mistake is resetting one half of a two-part system.

```
# Fresh chain, redeployed contracts, reseeded database.
./scripts/reset_demo.sh
```

D2. Common problems

Symptom	Cause and fix
pip fails with a long Rust error mentioning PyO3	You are on Python 3.14. Install 3.11-3.13, delete backend/.venv, and run start.sh again.
“Nothing is under seal right now” on the dashboard	There is no sealed paper — either it has been released, or none was made. Assemble one in Paper Builder, encrypt it, then register it with a release time.
Sealing a paper fails with PaperAlreadyRegistered	The database was reset without the chain. Run <code>./scripts/reset_demo.sh</code> .
Everything logs you out at once	The database was reseeded, so the accounts are new. Sign in again.
The site loads as a blank dark page	A production build was run while the development server was live, and they share a folder. Stop the site, delete frontend/.next, and start again.
A port is already in use	Something is still running from last time. <code>./scripts/stop.sh</code>
e2e_demo.py reports “Seeded near-duplicates detected”	The question pool has already been partly used by an earlier paper, so the run had fewer questions to choose between. Run <code>./scripts/reset_demo.sh</code> first. The demonstration needs a full pool; seal your demo paper <i>after</i> the end-to-end run, not before.
The seal shows “chain unreachable”	The blockchain is not running. This is the honest failure mode, not a bug: release is refused rather than falling back to a server clock. Start it with start.sh.

D3. Stopping everything

```
./scripts/stop.sh
```

This leaves the database and the chain state on disk, so starting again resumes exactly where you left off. Use `reset_demo.sh` instead when you want to begin from a clean slate.

D4. Where things live

Path	What it holds
backend/	The API, encryption, permissions, audit chain, seed data
blockchain/	The two smart contracts and their tests
frontend/	The web app
scripts/	Startup, reset and the end-to-end demonstration
docs/	This manual and the from-scratch explanation
backend/securelock.db	The demo database (safe to delete via <code>reset_demo.sh</code>)
.logs/	Output from services started by <code>start.sh</code>